

Guia tecnica para developer

Justificacion tecnica, lineamientos de correccion y prompts para trabajar el PR con un LLM sin romper fixtures, datos ni migraciones.

Repositorio

erpnext-bel-urbanizacion

Base

master

Head

DiegoDeveloper

Objetivo del documento

Este documento explica los hallazgos tecnicos del PR #7 hacia `master` del modulo privado `urbanizacion`, y entrega guias/prompts para que el developer, usando un LLM, pueda corregir el PR sin romper datos, migraciones ni reglas de seguridad de Frappe/ERPNext v15.

PR revisado:

- Titulo: `Prevent duplicate follow-ups for contracts and enhance CartaReserva`
- Base: `master`
- Head: `DiegoDeveloper`
- Archivos modificados:
 - `urbanizacion/fixtures/client_script.json`
 - `urbanizacion/fixtures/dctype.json`
 - `urbanizacion/fixtures/print_format.json`
 - `urbanizacion/fixtures/server_script.json`

Resumen ejecutivo

El PR agrega una mecanica de confirmacion para `ContratoVenta` y `CartaReserva`, cambia la notificacion del primer desembolso para ejecutarse al confirmar, permite editar `monto_financiar`, y elimina el campo `peso` de `Actividad0bra` junto con su uso en el formato de impresion de `Seguimiento0bra`.

La intencion funcional es razonable, pero el PR no debe mezclarse tal como esta porque introduce tres riesgos:

1. Elimina un campo de datos (`Actividad0bra.peso`) que existe actualmente y tiene valores cargados.
2. La accion de confirmar depende del cliente/UI y no tiene suficiente control server-side por rol.
3. Los nuevos campos `confirmado` estan exportados con metadata incompleta dentro del fixture de DocType.

Principios tecnicos aplicables en Frappe/ERPNext v15

1. No eliminar campos con datos sin migracion explicita

En Frappe, un DocType exportado como fixture representa el modelo que se aplicara durante `bench migrate` o instalacion de la app. Si un campo deja de existir en el fixture, el sitio puede quedar con diferencias entre metadata, tabla fisica y comportamiento de formularios/reportes.

Aunque Frappe no siempre borra inmediatamente la columna SQL, el campo deja de estar en el DocType y puede perderse de la UI, reportes, print formats, permisos y logica de negocio.

Regla practica:

- Si el campo todavia puede tener valor de negocio, conservarlo.
- Si se elimina por decision funcional, documentar la razon y agregar una migracion controlada o un plan de respaldo.
- Nunca retirar un campo solo porque un formato de impresion ya no lo muestra.

Hallazgo actual:

- En `testing15`, `tabActividad0bra` tiene `92` filas.
- `2` filas tienen `peso != 0`.
- Por tanto, eliminar `Actividad0bra.peso` implica perdida funcional de informacion ya cargada.

2. Las acciones sensibles deben validarse server-side

En Frappe, un Client Script mejora la experiencia de usuario, pero no debe ser la unica capa de seguridad. Cualquier usuario con permisos de escritura podria modificar campos mediante API, scripts, importaciones o consola si no existe validacion del lado servidor.

La confirmacion de `ContratoVenta` o `CartaReserva` parece una accion de negocio. Si genera correo, cambia estado operacional o dispara seguimiento, debe tener validacion server-side.

Regla practica:

- Client Script: boton, confirm dialog, indicadores visuales.
- Server Script o Python controller: validar permisos, detectar transiciones, ejecutar efectos secundarios.
- El servidor debe impedir `confirmado = 1` si el usuario no tiene rol autorizado.

Rol recomendado:

- `Urbanizacion Manager`
- Opcionalmente `System Manager` solo si se acepta administracion total.

3. Los fixtures deben exportarse completos y reproducibles

Los fixtures son la forma de convertir cambios hechos en Desk a codigo instalable. Para DocTypes, especialmente campos, es mejor que el fixture mantenga metadata completa y consistente, no entradas minimas.

Un campo exportado de forma incompleta puede funcionar en algunos casos, pero es fragil porque faltan atributos que Frappe normalmente espera en `DocField`:

- `parent`
- `parentfield`
- `parenttype`
- `idx`
- `read_only`
- `permlevel`
- `in_list_view`
- `in_filter`
- `print_hide`
- `search_index`
- otros flags propios del framework

Regla practica:

- Crear/ajustar el campo desde Desk o fixture source correcto.
- Ejecutar export de fixtures desde bench.
- Revisar diff para confirmar que el campo quedo completo.

4. Server Scripts en fixtures son instalables, pero requieren disciplina

Este modulo privado exporta Server Scripts en fixtures. Eso es valido para este caso, pero se debe controlar:

- Que los scripts no dependan solo de UI.
- Que no traguén errores críticos con `except Exception: pass` sin log.
- Que no creen duplicados al correr `After Save` repetidamente.
- Que validen transiciones usando `doc_before_save` o `doc.get_doc_before_save()`.
- Que no envíen correos múltiples por guardar el mismo documento varias veces.

Cambios requeridos antes de merge

Cambio 1 - Conservar o migrar `Actividad0bra.peso`

Recomendacion preferida:

- Mantener el campo `Actividad0bra.peso` en `doctype.json`.
- Si no se quiere mostrar en el formato impreso, retirar solo la columna del Print Format, no el campo del DocType.

Alternativa si negocio confirma que el campo ya no se usa:

- Crear documento de decision.
- Exportar datos existentes antes de eliminar.
- Agregar patch/migracion que archive o transfiera valores antes de retirar el campo.

Decision recomendada ahora:

- No eliminar `peso` en este PR.

Cambio 2 - Agregar validacion server-side para `confirmado`

La confirmacion debe bloquearse si el usuario no tiene rol autorizado.

Ejemplo conceptual para Server Script `Before Save` en `ContratoVenta` y/o `CartaReserva`:

PYTHON

```
before_doc = doc.get_doc_before_save()
was_confirmed = bool(before_doc.confirmado) if before_doc else False
is_confirming = bool(doc.confirmado) and not was_confirmed

if is_confirming:
    allowed = frappe.has_role("Urbanizacion Manager") or frappe.has_role("System Manager")
    if not allowed:
        frappe.throw("Solo Urbanizacion Manager puede confirmar este documento.")
```

Notas:

- La validacion debe vivir en servidor.
- El Client Script puede ocultar o mostrar el boton, pero no basta.
- Si `System Manager` no debe poder operar negocio, removerlo de `allowed`.

Cambio 3 - Reexportar correctamente `confirmado`

Los campos `confirmado` en `ContratoVenta` y `CartaReserva` deben quedar como `DocField` completos dentro del fixture de `DocType`.

Recomendacion:

1. Aplicar campos en un sitio de desarrollo/testing.
2. Usar export de fixtures con bench.
3. Revisar que el diff no contenga un objeto minimo de solo 5 propiedades.
4. Confirmar que `bench migrate` funciona en `testing15` antes de llevar a produccion.

Cambio 4 - Evitar correos duplicados con transicion real

El PR ya intenta esto con `doc_before_save`, lo cual va en la direccion correcta. Aun asi, debe quedar claro que el correo se envia solo cuando:

- `confirmado` pasa de `0` a `1`.
- El documento tiene desembolsos.
- Hay destinatarios validos.
- El servidor acepta el rol que confirma.

Evitar enviar correo si:

- El documento ya estaba confirmado.
- Se guarda por otro cambio no relacionado.
- El usuario no tiene permiso de confirmacion.

Prompts recomendados para el LLM del developer

Prompt 1 - Analizar el PR sin modificar codigo

TEXT

Actua como reviewer senior de Frappe/ERPNext v15. Revisa este PR del modulo privado urbanizacion contra master. No mezcles ni edites todavia. Identifica riesgos de migracion, perdida de datos, fixtures incompletos, Server Scripts inseguros, permisos por rol y efectos secundarios por After Save. Devuelve findings ordenados por severidad con archivo y campo afectado.

Prompt 2 - Corregir eliminacion accidental de `ActividadObra.peso`

TEXT

En el PR actual se elimino el campo `ActividadObra.peso` del fixture `urbanizacion/fixtures/doctype.json`. Ese campo existe en master y hay datos cargados en `testing15`, por lo que no debe eliminarse en este PR. Restaura el DocField `peso` exactamente como esta en master. Si el formato de impresion de SeguimientoObra ya no debe mostrar `Peso`, deja solamente el cambio del `Print Format`. No borres columnas ni datos. Al final muestra el diff limitado a `doctype.json` y `print_format.json`.

Prompt 3 - Agregar validacion server-side para confirmar

TEXT

Implementa validacion server-side para que el campo `confirmado` de `ContratoVenta` y `CartaReserva` solo pueda cambiar de `0` a `1` por usuarios con rol `Urbanizacion Manager`. Si se decide incluir `System Manager`, dejalo explicito y documentado. La validacion debe ejecutarse en servidor, no solo en Client Script. Usa `doc.get_doc_before_save()` o `doc_before_save` para detectar transicion real. No envies correos si la validacion falla. Devuelve el patch y explica donde queda la proteccion.

Prompt 4 - Reexportar fixture de DocType correctamente

TEXT

Los campos `confirmado` en `ContratoVenta` y `CartaReserva` estan exportados con metadata incompleta. Reexporta los DocTypes desde `Frappe/bench` para que los DocField `confirmado` queden completos como los demas campos del fixture. No cambies otros DocTypes salvo que sea consecuencia real del export. Revisa que `urbAmbiente.cantidad` siga siendo `Float`. Muestra un resumen del diff y valida JSON.

Prompt 5 - Validar que no se rompa `migrate`

TEXT

Antes de proponer merge, valida que el modulo urbanizacion pueda migrar en testing15. Ejecuta los checks de JSON, sintaxis de Server Scripts, sintaxis basica de Client Scripts, y `bench --site testing15.inversionesbel.com migrate`. Luego confirma que: `urbAmbiente.cantidad` es Float, importar-`lotes` sigue limitado a Urbanizacion Manager, no hay duplicados en DocType Link, y `ActividadObra.peso` sigue definido si no fue aprobado eliminarlo.

Prompt 6 - Preparar commit enfocado

TEXT

Haz un commit pequeno y enfocado para esta correccion del PR. El mensaje debe describir una sola tarea. No mezcles cambios de seguridad con cambios de formato si pueden separarse. Antes del commit, muestra `git status` y `git diff --stat`. Despues del commit, muestra el hash.

Checklist tecnico antes de merge

Ejecutar desde `/home/frappe/frappe-bench/apps/urbanizacion` como usuario `frappe`.

1. Estado Git

BASH

```
git status -sb
git diff --stat origin/master...origin/DiegoDeveloper
git diff --name-status origin/master...origin/DiegoDeveloper
```

2. Validar JSON de fixtures

BASH

```
python3 -m json.tool urbanizacion/fixtures/doctype.json >/dev/null
python3 -m json.tool urbanizacion/fixtures/client_script.json >/dev/null
python3 -m json.tool urbanizacion/fixtures/server_script.json >/dev/null
python3 -m json.tool urbanizacion/fixtures/print_format.json >/dev/null
```

3. Confirmar campos criticos

BASH

```
jq -r '[] | select(.doctype=="DocType") | .name as $dt | (.fields // [])[] | select(.fieldname=="cantidad" or .fieldname=="confirmado" or .fieldname=="peso" or .fieldname=="monto_financiar") | [$dt, .fieldname, .fieldtype, (.hidden|tostring), (.read_only|tostring), (.label // "")] | @tsv' urbanizacion/fixtures/doctype.json
```

Valores esperados:

- `urbAmbiente.cantidad` debe seguir como `Float`.
- `ActividadObra.peso` debe seguir existiendo salvo decision formal de eliminarlo.
- `ContratoVenta.confirmado` debe ser `Check`, oculto si asi se desea.
- `CartaReserva.confirmado` debe ser `Check`, oculto si asi se desea.

4. Revisar datos existentes antes de eliminar campos

BASH

```
cd /home/frappe/frappe-bench
bench --site testing15.inversionesbel.com mariadb <<'SQL'
SHOW COLUMNS FROM `tabActividadObra` LIKE "peso";
SELECT COUNT(*) AS total_actividad,
       SUM(CASE WHEN peso IS NOT NULL AND peso != 0 THEN 1 ELSE 0 END) AS con_peso
FROM `tabActividadObra`;
SQL
```

Si `con_peso > 0`, no eliminar el campo sin migracion o aprobacion explicita.

5. Validar migracion en testing

BASH

```
cd /home/frappe/frappe-bench
bench --site testing15.inversionesbel.com migrate
bench --site testing15.inversionesbel.com clear-cache
```

6. Validar metadata despues de migrate

BASH

```
cd /home/frappe/frappe-bench
bench --site testing15.inversionesbel.com mariadb <<'SQL'
SELECT parent, fieldname, fieldtype, hidden, read_only
FROM `tabDocField`
WHERE parent IN ("ContratoVenta", "CartaReserva", "ActividadObra", "urbAmbiente")
  AND fieldname IN ("confirmado", "peso", "cantidad", "monto_financiar")
ORDER BY parent, idx;
SQL
```

Recomendacion final para PR #7

No mezclar el PR #7 hasta corregir los puntos anteriores.

Orden recomendado de correccion:

1. Restaurar `Actividad0bra.peso` o documentar eliminacion con migracion.
2. Agregar validacion server-side por rol para `confirmado`.
3. Reexportar correctamente los campos nuevos como fixtures completos.
4. Validar `bench migrate` en `testing15`.
5. Hacer commits pequenos por tarea.
6. Solo despues considerar merge a `master`.